Lecture 16 - C-Store

Setup

Paper today

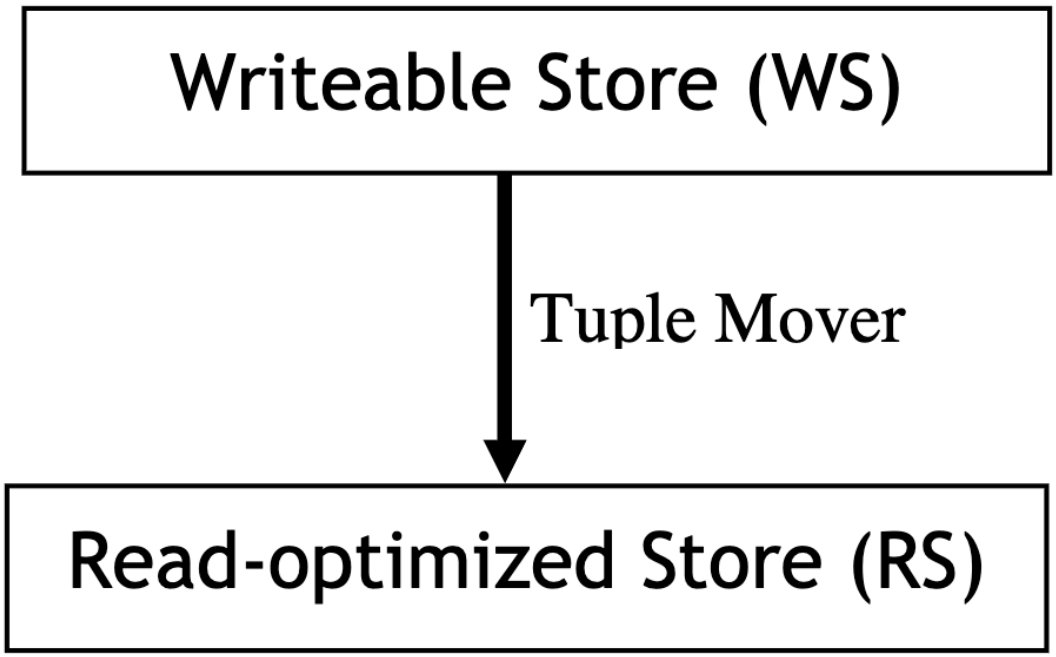
- **C-Store: A Column-Oriented DBMS**, VLDB 2005

Some history:

- By the mid-2000s, the traditional row-store DBMS had "won" the commercial market
  - IBM DB2, Oracle, SQL Server all used the "N-ary" storage model (NSM): one disk page holds multiple full rows
  - Write-optimized by design — inserts and updates touch a single page per row
  - Worked well for OLTP (many small writes, point lookups)
- Meanwhile, data warehousing and analytics workloads were growing:
  - TPC-H and similar benchmarks — scans over millions of rows, aggregating a few columns at a time
  - These workloads touch 2–5% of a table's columns per query on average
  - Q: Why is NSM a bad fit here?
    - A full row is read from disk even when only a few columns are needed — massive wasted I/O
    - Compression is hard when row data is heterogeneous
    - SIMD and vectorized processing hard to exploit when operating on rows
- C-Store proposes a clean-slate re-design: optimize for reads, separate the write path, compress aggressively
- This work directly led to **Vertica**, and influenced MonetDB (future versions), Amazon Redshift, Snowflake, DuckDB, and many others
- Will study other variants of column stores in subsequent lectures

C-Store Intro

- Main contributions of C-Store
  - Storing data by column rather than row
  - Columnar compression and packing
    - Basically storing values at byte or word boundaries is wasteful
      - If we know that cardinality is smaller, can "densepack"
    - And can also apply encoding to represent values more compactly
    - e.g., US state can be stored as 6 bits, plus a mapping - and six bits can be stored compactly
    - Can go further and perform compression -- but can lead to more computation
      - Paper mentions CPU is faster than disk bandwidth - both have gotten faster, but sophisticated compression schemes are less likely to be valuable given improvements in disk bandwidth
  - An overlapping set of column projections (perhaps least relevant feature of C-Store today)
    - Each column can appear in multiple projections
    - Each projection is sorted by a set of attributes
    - Still want to be able to recover the original relation through joins
  - Supporting writes through a Write Store and reads via a Read Store, connected through a tuple mover



    -
    - Compaction that moves pointwise, row-level edits in the write store to the RS.
  - Txns via SI and stale snapshot reads

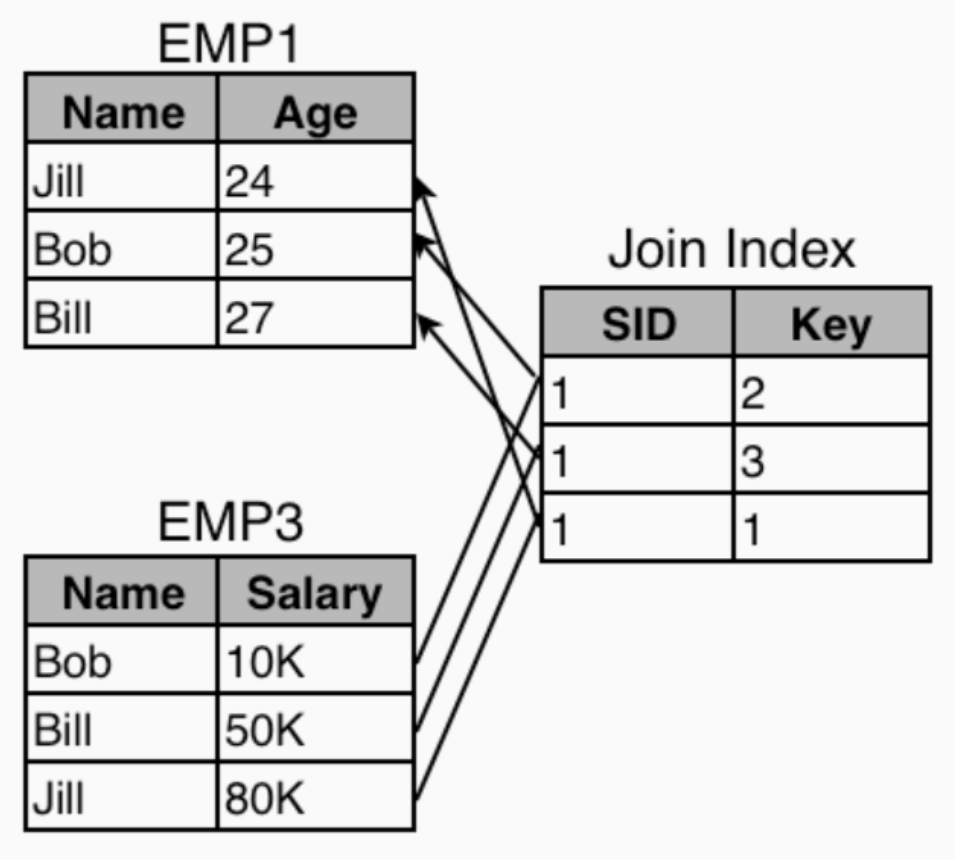
> *"It should be emphasized that while many of these topics have parallels with things that have been studied in isolation in the past, it is their combination in a real system that make C-Store interesting and unique."*

Detour to cover Bitmap Indexes

- Essentially, a bitmap per distinct value of a given attribute
  - Mainly work well for low-cardinality columns
  - e.g columns with few distinct values (gender, status, region, boolean flags)
- Can also compress very well with RLE
- Identifying which tuples match a combination of predicates are essentially bitwise ops, very efficient
- Commonly used in data warehouses and OLAP systems (like Oracle, Redshift) for multi-column filter queries.
- Downsides:
  - Don't work for high cardinality attributes
    - But see Roaring Bitmaps
  - Write-heavy workloads suffer

Data Model

- As mentioned, overlapping set of column projections
  - EMP1(name, age| age)
  - EMP2(dept, age, DEPT.floor| DEPT.floor)
  - EMP3(name, salary| salary)
  - DEPT1(dname, floor| floor)
- Can include foreign keys from other relations
- Physically, each projection partitioned into segments:
  - each segment has an ID
  - each row in segment has an implicit rowID, called *storage key* (reconstructed from position)

- To support joins there are so called *join indexes*
  - mapping between one set of segments in a projection to another
  - 

- Q: How is this different from a materialized view or an index?
  - Closer to a materialized view, but the whole data model is built around projections — they are not auxiliary.
  - There is no primary copy.
- Q: What is the cost of having many projections with different sort orders?
  - Join indexes become large, and keeping them consistent is non-trivial, especially during the Tuple Mover merge process

Read Store

- Detour for data compression:
  - Q: What is data compression?
    - Approximate compression
    - Exact compression
  - Overall a tradeoff between compression factor and latency for compression
  - Typical Exact Compression Schemes
    - Dictionary Encoding
    - Run Length Encoding
    - Huffman Coding
      - Initialize all symbols as leaves
      - Bottom up, merge two children with smallest counts
    - Lempel Ziv Welch Coding
      - Exploits repeated sequences
      - Uses backreferences
- Compression based on:
  1. **Ordering**: is the column sorted by its own values (**self-order**) or by another column's values (**foreign-order**)?

2. **Cardinality**: does the column have **few** or **many** distinct values?

- Encoding schemes:
  - Self-order (i.e., ordered by that column), few distinct values:
    - (v, f, n) such that v is a value stored in the column, f is the position in the column where v first appears, and n is the number of times v appears in the column
    - Q: What queries benefit most from Type 1 encoding?
      - GROUP BY or predicate filters on the sort key — you can binary search for the run and skip the rest
  - Foreign-order, few distinct values
    - (v, b), where b is a bitmap
    - b is further run-length encoded: Q: what is RLE?
    - Q: What is the trade-off vs. Type 1?
      - Larger than Type 1 because runs are broken by the foreign sort order, but still much smaller than raw storage
  - Self-order, many distinct values
    - store deltas from previous value instead of actual values
    - Q: Why is delta encoding appropriate here rather than RLE?
      - Values are mostly distinct, so runs are length 1. Delta exploits the monotonicity of sorted data instead.
  - Foreign-order, many distinct values
    - not much can be done

Write Store

- Also a column store, same design as RS
  - But no compression, store data directly
  - Use a B+tree to maintain logical sort order
- But explicitly store a storage key -
  - in RS, this corresponds to implicit sort order, here it corresponds essentially to a tuple ID
  - Instantiated to be larger than existing storage keys across all segments
  - Each columnar value is stored as *(v, sk)* - with an index on *sk*, so can retrieve all values of a tuple across all projections using this
- Additionally store, for each projection, *(s, sk)* - with *s* being a sort key value, and *sk* first storage key where *s* appears
- Now to reconstruct a given projection, need to take into account both the WS and RS
- Umm, complicated!

> *Join indexes can now be fully described. Every projection is represented as a collection of pairs of segments, one in WS and one in RS. For each record in the "sender," we must store the sid and storage key of a corresponding record in the "receiver." It will be useful to horizontally partition the join index in the same way as the "sending" projection and then to co-locate join index partitions with the sending segment they are associated with. In effect, each (sid, storage key) pair is a pointer to a record which can be in either the RS or WS.*

Transaction Model

- Updates lead to new tuples inserted at all projections in the WS, with same storage key
  - Downside: each tuple insertion needs to make changes to several pages, at least as many as number of projections
    - But since WS is small, mostly all memory-resident
- Reads:
  - want it to be between a *high water mark* (not too new to conflict with writes)
  - and a *low water mark* (not too old that we need arbitrary time-travel)

- WS tracks epochs for updates/deletes
  -

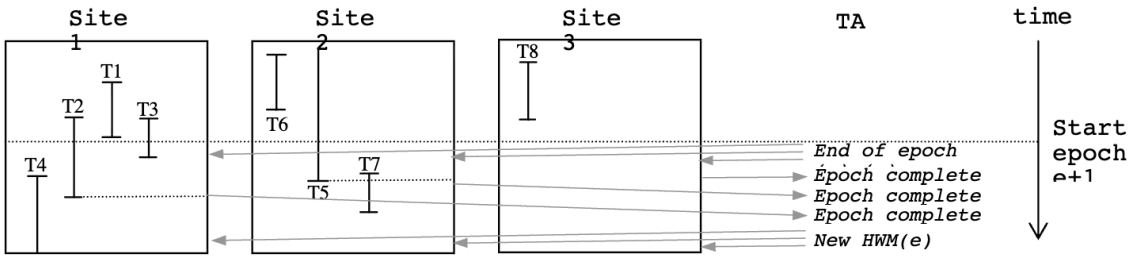


Figure 3. Illustration showing how the HWM selection algorithm works. Gray arrows indicate messages from the TA to the sites or vice versa. We can begin reading tuples with timestamp *e* when all transactions from epoch *e* have committed. Note that although T4 is still executing when the HWM is incremented, read-only transactions will not see its updates because it is running in epoch *e*+1.

  - Basically when the coordinator says end of epoch, individual nodes respond back when the txns still active when message is received have all finished
  - This allows to set a new HWM
    - Guarantee: no RS writes have happened after LWM (so RS doesn't need to worry about epochs)
- RW txns use Strict 2PL
- UNDO logging, 2PC, more details in 6.2 and 6.3 (skipped in interest of time)

Tuple Mover

- Essentially a "merge" of the WS and RS for each projection
- Do so for everything up to the LWM
- Creates a new merged set of RS segments that replaces the old one
- Increase LWM appropriately
  1. Takes a block of tuples from a WS segment
  2. Sorts them appropriately for the corresponding RS projection
  3. Merges them into the RS segment (append + re-sort or batch bulk load)
  4. Updates all relevant join indexes
  5. Removes the corresponding tuples from WS

Query Optimizer

- Pretty sketchy details ... not much here

Evaluation

- Looks pretty good! Better than a row store and a column store
  - But beware: projections have been carefully selected to match the workload
  - Anticipating this concern, they also implement materialized views for other systems based on the projections
    - But the other systems are not able to take advantage of the MVs
- Overall, compression and reading less data can be a huge win